

Milestone 3: Reflections and Model

Yucai Zhong and Christina McCallum and Jennifer Flake and Rachelle De Jager

1 Reflection Journal

1.1 Yucai Zhong’s Reflection

From this shared task, I expected to gain two things. First, I wanted to better understand how membership inference works in practice instead of only reading about it conceptually. Second, I wanted to contribute a method that was not just a copy of the milestone 1 baseline, but still simple enough to explain and implement well. My task focused on comparing the 135M and 360M language models on the same discharge notes, keeping the original features from both models and adding cross-model delta features such as the differences in loss, perplexity, entropy, and confidence statistics. Through this process, I gained a more concrete understanding of feature engineering, model comparison, and how small design choices can affect downstream classification performance. I also learned that relative disagreement between two models can provide useful signals beyond absolute confidence values alone.

The task went fairly well overall, although not without difficulties. One part that went well was that the idea remained focused throughout the implementation: instead of adding many new metrics, I built on the baseline features and introduced a clear extension based on cross-model differences. This made the method easier to justify. Another positive point is that the method produced reasonable validation results, which suggests that the direction is meaningful. At the same time, not everything went smoothly. The biggest barrier was the high cost of feature extraction, since the pipeline needed to run both the 135M and 360M models and compute multiple statistics for each sample. This made experimentation slower and limited how many variations I could test. Another challenge was that a good validation result does not guarantee the same level of improvement on the final hidden test set. Even so, I do feel that I am on the

right track because the method is coherent, reproducible, and supported by empirical results rather than intuition alone.

If I could restart this task, I would keep several good practices. I would still begin with an interpretable idea that extends the baseline in a controlled way. However, I would improve the experimental workflow. I would save intermediate outputs more systematically and plan the ablation process before running expensive computations. I would also spend more time checking which engineered features contribute the most, instead of assuming that all added deltas are equally useful. Overall, I would keep the clarity and logic of this approach, but I would improve the efficiency and organization of the experimentation process.

1.2 Christina McCallum’s Reflection

After learning about LLMs this and last block, I was expecting that this shared task would be a good way to get hands-on experience implementing the concepts we’ve been discussing over the past few months. The project fulfilled these expectations, but also introduced me to Membership Inference Attack and the field of AI safety more broadly. Reading research papers about the latest developments in MIA, and implementing some of the strategies discussed by these engineers, was a useful exercise in learning model development and cutting-edge AI safety research.

I believe the task went well overall – we were able to finish the project with a high score on the test set, and, as a group, we implemented a variety of methods to get there. I think our team communicated well and was able to build on the baseline that Jennifer implemented in the first milestone, which gave us a strong foundation to explore other MIA strategies. For me, the biggest barrier was understanding how to implement optimization techniques: we ran the two fine-tuned models over all 50,000 samples in the training data for feature

extraction, which ended up being quite computationally intensive. That being said, our combination of features and classification models did yield good results in the end, so I do believe that we were on the right track.

If I could restart this task again, I'd want to dig deeper into optimization techniques so I could better implement computationally efficient strategies in my own model implementation. I'd also want to refine my ability to code using .py files from the beginning (I'm still learning coding best practices), rather than running the model in one notebook. That being said, I'd want to keep our team's communication style and baseline strategy, as I think these were both strengths throughout the project.

1.3 Jennifer Flake's Reflection

Going into this project, I expected to gain exposure to membership inference attacks as a concept, but I did not anticipate how much I would learn from the open-ended structure of the task. The freedom to choose our own methodology meant that I had to read primary literature to understand what approaches were worth trying, rather than following a prescribed path. This made the work more demanding but also more engaging. I came away with a much deeper understanding of how language model memorization can be detected, and of the tradeoffs between different attack strategies.

Overall, the project went well. Our models improved meaningfully across milestones, with each iteration building on the features and lessons of the previous one. Team collaboration was a genuine strength: we divided work effectively and were willing to incorporate each other's ideas, which is reflected in the final model being a synthesis of components developed across the group. The biggest barrier was not technical but organizational. Because we were working in Colab notebooks, our code and saved artifacts were distributed and hard to reproduce cleanly. Extracted features and trained models were not always preserved in a systematic way, which sometimes meant re-running expensive computations unnecessarily.

If I were starting over, I would establish a modular code structure from the beginning. Specifically, I would keep the notebook as a thin interface and store all logic in versioned Python scripts, so that swapping in a new methodology would only require replacing a file rather than restructuring the notebook. Our milestone 3 source code is close to being organized this way now, but I think it should

have been structured this way from the beginning. I would also implement a disciplined approach to saving extracted features and trained models from the start, with clear naming conventions. The collaborative practices and the habit of grounding design decisions in the literature and previous milestones are things I would carry forward without change.

1.4 Rachelle De Jager's Reflection

I expected to gain an understanding of Membership Inference Attack, both in theoretical terms through reading research papers, and in practical terms through attempting to implement the methods we explored. I discovered that I quite enjoyed this process of theoretical research alongside objective application. It quickly became clear that our capabilities absolutely did not match those of the researchers in terms of access to compute and time constraints.

I specifically learned that using a reference model as comparison is effective for clarifying the membership signal. With each week's milestone, I also learned that feature engineering has clear and practical benefit, rather than understanding it in theoretical terms only. While time was obviously spent on the ML modelling itself, I also spent a lot of time managing challenges around Colab sessions and runtime, saving to Google Drive to prevent loss of progress, and modularizing python scripts.

The task went fairly well in terms of exploration as the feature importance rankings guided steps taken in subsequent milestones. Collaboration with my team went very well. We communicated clearly and each made solid contributions. I found my team members to be reliable and motivated.

The biggest barrier was time and compute. In milestone 2 I wanted to implement part of the method from the Fu et al paper (NeurIPS 2024) proposing SPV-MIA. Unfortunately, it is very time and compute intensive, so I was only able to apply the method to a very small subset of the data, which did not provide a strong enough signal for the classifiers to pay it any attention.

We iteratively implemented a system with version saving strategies, which was a good practice. Given the chance to re-start the project, I would have refined this pipeline with my team earlier, with clearer jupyter notebooks for ease of model development. I would also have added a systematic hyperparameter search aspect to each week's

model, given more time. Lastly, I would look at ways to specifically manage GPU use, which remained low when I ran models in Colab, despite batching effort

2 Milestone 3 Model

We prepared two models, Model 5 and Model 6. Model 5 is a compilation of the most useful features from previous models submitted with the addition of some features, and Model 6 built on Model 5 with the addition of further features. Model 6 is our final submission and is detailed below.

2.1 Model description

For this milestone's model, we decided to combine the useful, effective parts of previous models together. In addition to combining previous model approaches, we added new features based on HT-MIA. By reading "What Hard Tokens Reveal: Exploiting Low-confidence Tokens for Membership Inference Attacks against Large Language Models" by Md Tasnim Jawad, Mingyan Xiao, and Yanzhao Wu, we learned about HT-MIA, which is an approach to MIA that compares token-level probability improvements by extracting features that capture the difference between the probabilities assigned by the fine-tuned model and the probabilities assigned by the pre-trained reference model on the hardest tokens. As the paper says, "The key intuition is that non-member samples behave similarly for both target (F) and reference (B) models, while the fine-tuned model F assigns systematically higher probabilities than the reference model B to member samples, especially at hard-to-predict tokens with low confidence."

Further additions are: - Min-K%++, normalized token probability features; - extended the HT-MIA features to include values of K = 40% and K = 50%; - token-level log-likelihood ratio features; and - weighted soft-voting ensemble across the classifiers.

The new HT-MIA features differentiate this model from all of the previous models our group has created. The HT-MIA features we added include `ht_mia_score_k10`, `ht_mia_score_k20`, and `ht_mia_score_k30`. On seeing that feature importance increased with K, we extended to include `ht_mia_score_k40` and `ht_mia_score_k50` in order to verify that we had not reached a plateau at K=30%. These scores are the fraction of the bottom-K% hardest tokens (ranked by the target

model's probability) where the target model beats the reference model. The range of scores are 0–1, with higher scores suggesting that the token is more likely a member. The five HT-MIA features we created only vary in K, with K=10, K=20, K=30, K=40 and K=50.

The token-level log likelihood ratio (LLR) features were also added. For each token, the LLR is calculated as the difference between the log-probability predicted by the fine-tuned model against that assigned by the reference model. This feature captures the magnitude of the advantage the fine-tuned model has over the reference model. Six LLR features were added: `llr_mean`, `llr_std`, `llr_min`, `llr_max`, `llr_k10`, and `llr_k30`. `llr_k10` and `llr_k30` are the mean LLR on the "hardest" 10% and 30% of tokens.

Min-K%++ is an improvement on existing Min-K% features. Existing features calculate the raw probability, while Min-K%++ normalizes those probabilities by the mean and standard deviation of the entire vocabulary distribution at that specific token position. A high score suggests that the model is confident on a token beyond what the position would naturally suggest. The bottom 10%, 20% and 30% of tokens are hardest to predict, so these features challenge the model even further, such that if the model is able to predict them accurately, this signals a likely member text. Min-K%++ features make the score more robust by taking into account text complexity that exists separately to whether the model was able to memorize or not. Three Min-K%++ features were added at K=10%, K=20% and K=30%.

The remaining features of the model have been used in previous models. They are: `perplexity`, `loss`, `mean_token_prob`, `min_token_prob`, `max_token_prob`, `std_token_prob`, `mean_entropy`, `top_k_mass`, `low_conf_ratio`, `min_k10_prob`, `min_k20_prob`, `min_k30_prob`, and `n_tokens`. These features have been discussed in previous milestone reports.

Continuing the practice of training multiple classifiers and then selecting the best one, we implemented a logistic regression, random forest, gradient boosting and XGBoost. However, we also added a weighted soft-voting ensemble feature that combines all four classifiers, each weighted by their respective validation AUC scores.

2.2 Implementation

In our implementation for this milestone, we minimized our use of Jupyter Notebooks. Our model6.ipynb file serves as an interface for Colab and is mostly used to call functions that exist in our accompanying .py script files. In addition, the model6.ipynb notebook performs basic functions like mounting the google drive and showing helpful runtime output. Aside from the notebook file, the source for this model includes the following files: batches.py, build_membership_classifier.py, compute_perplexity_and_features.py, models.py and config.py.

When model6.ipynb executes, it runs the following pipeline:

1. Setup. The notebook mounts Google Drive as a save directory and sets the compute device to GPU if available.

2. Data loading. load_data() fetches the train, validation, and test splits from HuggingFace.

3. Model loading. load_models_and_tokenizer() loads four models from HuggingFace: the fine-tuned target models (SmolLM2-135M and SmolLM2-360M) and their corresponding pre-trained reference models, along with a shared tokenizer.

4. Feature extraction. process_all_splits() runs feature extraction across all three splits and all four models. Each is saved immediately to the Google Drive upon completion to protect against runtime disconnection issues. For each text, compute_perplexity_and_features() computes perplexity, token-level probability statistics, Min-K% scores, Min-K%++ scores, HT-MIA scores and token-level log-likelihood ratio features. The reference models are passed in specifically to support the HT-MIA and LLR computations. The resulting features are saved to Google Drive via save_features() and can be reloaded in a later session with load_features() to skip re-extraction.

5. Classifier training. build_membership_classifier() combines the 135M and 360M feature sets, adds cross-model delta features, and trains four classifiers: logistic regression, random forest, XGBoost, and gradient boosting. Each is evaluated on the validation set by AUC and TPR at FPR=0.1. A soft-voting ensemble is then calculated by averaging predicted probabilities from each of the four classifiers and weighting them by the classifier's validation AUC score. The best-performing classifier or ensemble

is selected and saved to Google Drive.

6. Submission generation. generate_submission() applies the best classifier to the test set and writes the membership predictions to a CSV file, which is then downloaded from Colab. If the best classifier is the ensemble, prepare_test_predictions() generates predictions from each classifier and averages them.

2.2.1 Here is a summary of the functions in the code base:

Function: compute_perplexity_and_features()

File: compute_perplexity_and_features.py

Purpose: Computes perplexity, token-level probability statistics, Min-K% scores, Min-K%++ scores, HT-MIA scores and token-level LLR features for a single input text.

Parameters: model, tokenizer, text, max_length=512, ref_model=None

Returns: dict of scalar features

Called by: analyze_sample_batch()

Function: analyze_sample_batch()

File: batches.py

Purpose: Extracts features for a batch of texts using a single model.

Parameters: model, tokenizer, texts, labels, model_name, ref_model=None

Returns: feature DataFrame

Called by: process_dataset_in_batches()

Function: process_dataset_in_batches()

File: batches.py

Purpose: Runs feature extraction over a full dataset in batches.

Parameters: model, tokenizer, texts, labels, model_name, batch_size=1000, ref_model=None

Returns: aggregated DataFrame

Called by: process_all_splits()

Function: process_all_splits()

File: batches.py

Purpose: Coordinates feature extraction across all train/val/test splits for both models.

Parameters: model_135m, model_360m, ref_135m, ref_360m, tokenizer, splits, batch_size=500

Returns: dict of 6 DataFrames

Called by: model6.ipynb

Function: save_features()

File: batches.py

Purpose: Saves extracted feature DataFrames to

disk.

Parameters: save_dir, features

Returns: None

Called by: model6.ipynb

Function: load_features()

File: batches.py

Purpose: Loads previously saved feature DataFrames from disk.

Parameters: save_dir

Returns: dict of DataFrames

Called by: model6.ipynb

Function: _build_feature_matrix()

File: build_membership_classifier.py

Purpose: Combines 135m and 360m feature sets and computes cross-model delta features.

Parameters: df_135m, df_360m

Returns: combined feature DataFrame

Called by: build_membership_classifier(), prepare_test_predictions()

Function: compute_zlib_baseline()

File: build_membership_classifier.py

Purpose: Computes zlib compression ratio as a baseline membership inference signal.

Parameters: val_df

Returns: dict (auc, tpr)

Called by: model6.ipynb

Function: build_membership_classifier()

File: build_membership_classifier.py

Purpose: Trains and evaluates membership inference classifiers on the combined feature set and computes a weighted soft-voting ensemble from all four base classifiers.

Parameters: train_135m, train_360m, val_135m, val_360m

Returns: dict of classifier results

Called by: model6.ipynb

Function: prepare_test_predictions()

File: build_membership_classifier.py

Purpose: Generates membership probability scores for the test set using a trained classifier. If the best classifier is the soft-voting ensemble, generates predictions from each base classifier and averages them, weighted by validation AUC score.

Parameters: test_135m, test_360m, best_classifier

Returns: np.ndarray

Called by: generate_submission()

Table 1: Results by Classifier

	AUC	TPR@FPR=0.1
Method: Loss		
Ensemble (Weighted)	0.9487	0.8310
XGBoost	0.9470	0.8302
Gradient Boosting	0.9447	0.8232
Random Forest	0.9399	0.8284
Logistic Regression	0.9336	0.8272
Method: Zlib Entropy		
—	0.5075	0.1028

Function: generate_submission()

File: build_membership_classifier.py

Purpose: Produces the final submission CSV from test set predictions.

Parameters: test_135m, test_360m, best_classifier, test_df, output_path

Returns: DataFrame (id, is_member)

Called by: model6.ipynb

Function: get_model_dtype()

File: models.py

Purpose: Selects the optimal floating point dtype (bf16 or fp32) based on the device.

Parameters: device

Returns: torch.dtype

Called by: load_models_and_tokenizer()

Function: load_models_and_tokenizer()

File: models.py

Purpose: Loads the fine-tuned target models, reference models, and shared tokenizer.

Parameters: device

Returns: tuple (tokenizer, model_135m, model_360m, ref_model_135m, ref_model_360m)

Called by: model6.ipynb

Function: load_data()

File: models.py

Purpose: Loads the train, validation, and test splits from HuggingFace.

Parameters: —

Returns: dict of DataFrames

Called by: model6.ipynb

2.3 Results

2.4 Discussion

All five classifiers operate on the same feature set, which combines per-model base features (perplexity, loss, token probability statistics, entropy) from

both the 135M and 360M models, cross-model delta and ratio features, Min-K% scores, Min-K%++ scores, HT-MIA scores and token-level LLR features. The feature matrix therefore captures both the absolute behaviour of each model and how the two models differ in their confidence on a given text.

Logistic Regression fits a linear decision boundary over the feature set. It is the simplest of the five classifiers and assumes that membership can be determined by a weighted sum of the input features. It achieved an AUC of 0.9336 and a TPR@FPR=0.1 of 0.8272. The competitive performance of logistic regression relative to the tree-based methods suggests that the feature space has strong linear structure and that the engineered features (particularly ratios and deltas) already capture much of the signal without requiring nonlinear interactions. Its main limitation is that it cannot model interactions between features, which likely accounts for the modest gap relative to the tree-based methods.

Random Forest builds an ensemble of decision trees, each trained on a random subset of features and data, and averages their predictions. It achieved an AUC of 0.9399 and TPR@FPR=0.1 of 0.8284. Unlike logistic regression, it naturally captures nonlinear interactions. Its feature importance rankings show that llr_k10_360m and llr_k30_360m are the two dominant predictors, with ht_mia_score_k40_360m and loss_ratio also featuring prominently. The appearance of the new LLR and HT-MIA features confirm that these additions offer meaningful signal. Random forests can overfit on high-dimensional data and are generally less efficient than boosting methods, which may explain why it did not perform quite as well as XGBoost.

XGBoost uses gradient boosting with regularization to build an ensemble of trees sequentially, each correcting the residual errors of the previous. It achieved the best individual AUC of 0.9470. Its feature importance is notably concentrated: llr_k10_360m alone accounts for nearly 20% of the total importance, and llr_k10_360m contributes a further 13%. Together, the two LLR features account for over 32% importance, which is more than all the HT-MIA features combined. This concentration suggests XGBoost is efficiently exploiting a small number of highly predictive features.

Gradient Boosting is similar to XGBoost but does not have the same regularization and parallelization optimizations. It achieved an AUC of

0.9447. Its feature importance is even more concentrated than XGBoost: llr_k10_360m alone, accounts for 64% of total importance and loss_ratio contributes a further 12%. This extreme concentration may indicate that gradient boosting is over-relying on a small subset of features and not using the rest of the feature set as much as possible.

Weighted Soft-Voting Ensemble combines the predictions of all four classifiers, weighting each by its validation AUC. It achieved the best overall AUC of 0.9487 and TPR@FPR=0.1 of 0.8310, which outperformed every individual classifier. Each of the four classifiers make errors that are somewhat smoothed out when combined.

Despite being theoretically motivated and appearing as a feature in several Membership Inference Attack papers, the Min-K%++ features did not appear in the top features for any of the classifiers. This is likely because the LLR features capture a similarly normalized signal with stronger input, and LLR also corrects for position difficulty by comparing to the reference model’s distribution for each token.

Error Analysis

Across all five classifiers, TPR@FPR=0.1 sits between 0.827 and 0.831, which means that when the false positive rate is held at 10%, approximately 1 in 6 true members is missed. The relatively small spread in TPR@FPR=0.1 across methods (less than 0.004) compared to the larger spread in AUC (0.9336-0.9487) suggests that the classifiers differ more in their overall ranking ability to classify members vs non-members, than they do at the 10% false positive data point.

The new LLR features dominated, suggesting that the magnitude of the fine-tuned model’s probability advantage over the reference, as captured by LLR, is a successful signal.

The consistent dominance of 360M model features over 135M features across all tree-based methods stands out. All top LLR and HT-MIA scores computed from the 360M model take the top positions in every importance ranking, while 135M HT-MIA scores appear lower in the importance ranking. This suggests that the larger model produces a cleaner memorization signal, which is what we expected because the larger capacity likely supports larger memorization.

The ordering of HT-MIA importance by K value does not follow a clear monotonic pattern. Larger K values do not consistently pro-

vide a stronger signal even though they examine more tokens with each increase in K . For example, `ht_mia_score_k40_360m` appears above `ht_mia_score_k30_360m` in some classifiers, but not all, while `ht_mia_score_k50_360m` contributes very little even though it examines the most tokens.

Potential Improvements

Hyperparameter tuning. Only XGBoost was tuned beyond defaults with one set of alternative hyperparameters. However this provided no benefit, producing the same AUC score as the default setup. Further systematic tuning of XGBoost (learning rate, tree depth, subsampling) could likely yield further AUC gains.

Gradient norm features. The gradient of the loss with respect to the input embeddings is another feature to explore. The idea is that member texts would show smaller gradient norms since the model has already seen those texts.

Feature selection. Given that gradient boosting assigns near-zero importance to the majority of the 55 features, aggressive feature selection or dimensionality reduction could reduce noise and potentially improve generalization, particularly for logistic regression.

Stacking. A meta-classifier trained on the four base classifiers could be applied, rather than a weighted average through ensembling. This method could support the model to learn when to extract more signal from one classifier over the others based on error patterns.

2.5 Kaggle results / observations

Our Kaggle score for the competition this week is 0.95105, which is an improvement over our model 5 submission at 0.93657, and last week's score of 0.90549.

3 Bibliography

Md Tasnim Jawad, Mingyan Xiao, and Yanzhao Wu. 2026. What Hard Tokens Reveal: Exploiting Low-Confidence Tokens for Membership Inference Attacks against Large Language Models. *arXiv preprint arXiv:2601.20885*.

Wenjie Fu, Huandong Wang, Chen Gao, Guanghua Liu, Yong Li, and Tao Jiang. 2024. Membership Inference Attacks against Fine-tuned Large Language Models via Self-prompt Calibration. [https://proceedings.neurips.cc/paper_files/paper/2024/file/f36ad694188bb4c4bbbd61e2038e069e-](https://proceedings.neurips.cc/paper_files/paper/2024/file/f36ad694188bb4c4bbbd61e2038e069e-Paper-Conference.pdf)

Paper-Conference.pdf. Advances in Neural Information Processing Systems, 37:134981–135010.

4 Contributions

4.1 Milestone 3

- Christina: Wrote personal reflection journal. .20
- Jennifer: Implemented and documented model 5. Submitted test results to Kaggle. Wrote personal reflection journal. .30
- Rachelle: Implemented model 6 and added documentation to Jennifer's model 5 documentation. Wrote personal reflection journal. .30
- Yucai: Wrote personal reflection journal. .20